

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

"FRANCISCO GARCÍA SALINAS"

UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA

INGENIERÍA DE SOFTWARE 4° "D"

- **Proyecto:** FashionFlow POS
- **Materia:** Análisis y Diseño Orientado a Objetos
- **Docente:** Cristian Boyain
- **Integrante 1:** Julio Miguel Sifuentes Sánchez
- **Integrante 2:** Ana Patricia Galvan Cruz
- **Versión:** 4.0 (Actualizada según guías académicas)
- **Fecha:** Junio de 2026

5. Diseño Orientado a Objetos

5.1 Jerarquía de Herencia — Productos

La clase abstracta **Producto** centraliza los atributos y métodos comunes. Cada departamento es una subclase que hereda de ella y puede extender sus propiedades según sus necesidades específicas.

- **Clase Base:** **Producto**
 - **Atributos:** sku: String, nombre: String, precio: Float, existencias: Int, departamento: String.
 - **Métodos:** obtener_info(): String, aplicar_descuento(p: Float): Float.

Clases derivadas por herencia:

1. **Calzado:** Añade talla: String, material: String.
 - **CalzadoDama:** Hereda de *Calzado*, añade tacon: Bool.
 - **CalzadoCaballero:** Hereda de *Calzado*, añade tipo: String.
2. **RopaDeportiva:** Añade deporte: String, genero: String.
3. **VestidoDeNoche:** Añade ocasion: String, tela: String.
4. **RopaHombre:** Añade talla: String.
5. **RopaMujer:** Añade talla: String.
6. **Accesorio:** Añade tipo_accesorio: String.

5.2 Diagrama de Clases Principal

Estructura y métodos base de las principales clases del sistema:

- **Producto:** sku, nombre, precio, stock | info(), descuento().
- **Inventario:** productos[], umbral | agregar(), reducir(), buscar().
- **Venta:** id, fecha, cajero, total | calcular(), cerrar().
- **DetalleVenta:** producto, cantidad, precio | subtotal().
- **Departamento:** nombre, productos[] | agregar(), listar().

- **Usuario:** id, usuario, rol | autenticar().
- **Reporte:** tipo, rango | generar(), exportar().

6. Casos de Uso (Sección Corregida)

6.1 Actores y Límites del Sistema

Para el modelado, el software **FashionFlow POS** define el límite del sistema bajo diseño. Se identifican exclusivamente los actores externos que interactúan directamente con los servicios del sistema para obtener un resultado observable de valor comercial. El software en sí o sus automatizaciones no se consideran actores secundarios.

Actor	Tipo	Descripción de la Intención / Objetivo
Cajero	Primario	Satisface el objetivo de procesar las transacciones de los clientes en caja y verificar la disponibilidad inmediata de las prendas o calzado.
Administrador	Primario	Satisface los objetivos de gestión estructural del catálogo, la configuración de departamentos y el análisis estratégico del rendimiento del negocio.

6.2 Catálogo de Casos de Uso

Se eliminan los casos de uso técnicos o de interfaz (como *Iniciar Sesión*), ya que fallan la **Prueba del Jefe** al no representar una tarea de negocio por sí misma. Los requerimientos de datos se unifican bajo la convención idiomática **"Administrar X"** y se nombran mandatoriamente iniciando con un verbo en infinitivo.

- **CU-01 Procesar Venta:** Registra las transacciones en el punto de venta agregando valor comercial directo (Pasa la prueba de Proceso Empresarial Elemental - EBP).
- **CU-02 Consultar Inventario:** Permite comprobar las existencias físicas de los artículos comerciales de la tienda.
- **CU-03 Administrar Productos:** Centraliza los comportamientos de altas, modificaciones y bajas del catálogo de mercancías.
- **CU-04 Administrar Departamentos:** Gestiona la categorización de los nuevos departamentos de ropa y calzado sin alterar las clases base existentes.
- **CU-05 Analizar Actividad Comercial:** Evalúa el rendimiento financiero y consolidado del negocio dentro de un periodo determinado.

6.3 Especificación Esencial y de Caja Negra

Las siguientes especificaciones describen únicamente las intenciones de los actores y las responsabilidades del sistema, omitiendo detalles de pantallas (UI) o especificaciones de bases de datos internas (como sentencias SQL).

ID y Caso de Uso	Actor	Descripción del Flujo (Responsabilidades del Sistema)
CU-01 Procesar Venta	Cajero	El sistema identifica los productos seleccionados mediante su SKU. El sistema calcula los subtotales, impuestos y el importe total aplicando las reglas de descuento correspondientes. Al confirmarse la transacción, el sistema registra la venta y deduce de forma consistente las existencias de los productos involucrados.
CU-02 Consultar Inventario	Cajero / Administrador	El sistema solicita los criterios de búsqueda (SKU o departamento). El sistema recupera y expone el estado de las existencias actuales correspondientes a la consulta.
CU-03 Administrar Productos	Administrador	El sistema procesa la incorporación, actualización de propiedades (precio, stock) o la inhabilitación de artículos del catálogo comercial, respetando las propiedades heredadas en la jerarquía de herencia.
CU-04 Administrar Departamentos	Administrador	El sistema procesa el alta o la edición de las categorías comerciales (ej. Calzado Caballero, Vestido Noche), permitiendo la asignación jerárquica de productos a sus respectivos módulos.

CU-05	Administrador	El sistema recopila los registros de operaciones del periodo solicitado. El sistema calcula las métricas consolidadas y genera un reporte estructurado de rendimiento para su uso analítico.
Analizar Actividad Comercial		

7. Arquitectura del Sistema — Patrón MVC

El sistema utiliza el patrón Modelo-Vista-Controlador (MVC) para separar las responsabilidades de datos, interfaz e interacciones, facilitando la modularidad y el mantenimiento.

- **Modelo:** Administra datos y lógica de negocio (Productos, Inventario, Usuarios, Reportes).
- **Vista:** Expone los elementos visuales esenciales al usuario (Pantallas funcionales correspondientes a Ventas, Inventario, Reportes y Control de Acceso).
- **Controlador:** Conecta la Vista con el Modelo, procesando operaciones lógicas como *Registrar venta*, *Actualizar inventario* y *Generar reportes*.

Principios SOLID Aplicados

- **S — Responsabilidad Única:** Cada clase tiene una sola razón de cambio; por ejemplo, la clase Venta solo se encarga del procesamiento de las transacciones comerciales.
- **O — Abierto / Cerrado:** Las nuevas variaciones de ropa o calzado se añaden como subclases directas sin modificar el comportamiento ni la estructura de la clase base Producto.
- **L — Sustitución de Liskov:** Cualquier subclase derivada (como CalzadoDama o RopaDeportiva) puede ser utilizada indistintamente en cualquier sección donde se requiera un objeto del tipo abstracto Producto.
- **D — Inversión de Dependencias:** Los controladores del sistema interactúan con abstracciones y contratos de los productos, impidiendo la dependencia directa de implementaciones acopladas.

8. Conclusión

FashionFlow POS es un sistema modular enfocado en optimizar el control de inventario y el proceso de venta de tiendas de calzado y textiles.

A través del uso estricto del análisis orientado a objetos, se ha desarrollado una estructura de herencia sólida que resuelve de manera limpia la diversidad de productos de la tienda. La depuración de la especificación de requisitos hacia casos de uso esenciales y de caja negra garantiza que el sistema esté diseñado en función del valor real del negocio y no amarrado a interfaces de usuario o tecnologías particulares, previniendo errores de diseño e incrementando la mantenibilidad a largo plazo. Este documento se consolida como la

especificación formal que guiará el desarrollo de la arquitectura de software, la codificación de controladores y el diseño posterior de interfaces.